

- 神经网络基础

- 人脑神经网络

神经元结构：

- 细胞体：通过改变电位产生细胞生理活动：兴奋和抑制
- 细胞突起：由细胞衍生出的部分
 - 树突：接受刺激
 - 轴突：传递刺激

神经元可以接收其他神经元的信息，也可以发送信息给其他神经元。神经元之间没有物理连接，并靠突触（Synapse）进行互联来传递信息。

- 人工神经网络

人工神经网络是为模拟人脑神经网络而设计的一种计算模型，它从结构、实现机理和功能上模拟人脑神经网络。每个节点代表一种特定函数，来自其他节点的信息经过其相应的权重综合计算，输入到一个激活函数中并得到一个新的活性值（兴奋或抑制）。从系统观点看，人工神经网络是由大量神经元通过极其丰富和完善的连接而构成的自适应非线性动态系统。

- 神经网络发展史

- 模型提出：1943~1969
- 冰河期：1969~1983，长年停滞及低潮
 - 反向传播算法发明 1974
- 反向传播复兴：1983~1995
 - Hopfield网络，联想记忆
 - 玻尔兹曼机，随机版Hopfield
 - 分布式并行+反向传播
 - 自动微分拓展反向传播
 - 梯度消失问题出现
- 流行度降低：1995~2006
 - 支持向量机&线性分类器压过，计算复杂性过高
- 深度学习复兴：2006~
 - 预训练+精调方法
 - 算力加强与科技公司加持

- 前馈神经网络

- 前馈网络

通过上一层的数据加权和辅以偏置作为神经元净输入 z

$$\begin{aligned} z &= \sum_{d=1}^D w_d x_d + b \\ &= \mathbf{w}^T \mathbf{x} + b \end{aligned}$$

净输入经过非线性激活函数获取神经元激活值 $a = f(z)$

- 常见激活函数及其缺点

- Logistic函数

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Tanh函数

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

■ ReLU函数

$$\begin{aligned} ReLU(x) &= \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases} \\ &= \max(0, x) \end{aligned}$$

■ LeakyReLU函数

$$\begin{aligned} LeakyReLU(x) &= \begin{cases} x & x \geq 0 \\ \gamma x & x < 0 \end{cases} \\ &= \max(0, x) + \gamma \min(0, x) \end{aligned}$$

■ Softplus函数

$$Softplus(x) = \log(1 + e^x)$$

○ 前馈网络计算流程

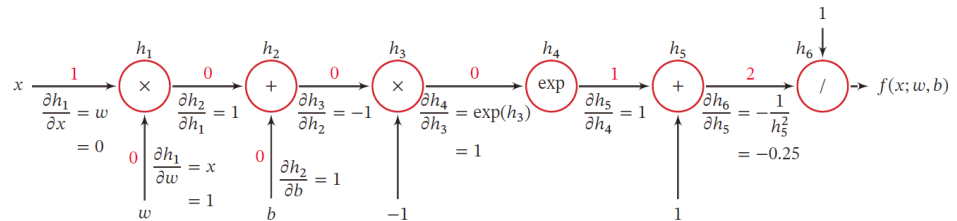
$$\begin{aligned} z^{(l)} &= \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \\ \mathbf{a}^{(l)} &= f_l(z^{(l)}) \end{aligned}$$

有以下一致性表达

$$\begin{aligned} \mathbf{a}^{(l)} &= f_l(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}) \\ \mathbf{a}^{(0)} &= \mathbf{x} \rightarrow \mathbf{a}^{(L)} = \varphi(\mathbf{x}; \mathbf{W}, \mathbf{b}) \end{aligned}$$

其中 $\mathbf{W}$, $\mathbf{b}$ 表示网络中所有层的链接权重及偏置

○ 自动微分计算图



● 卷积神经网络

○ 卷积层计算方法

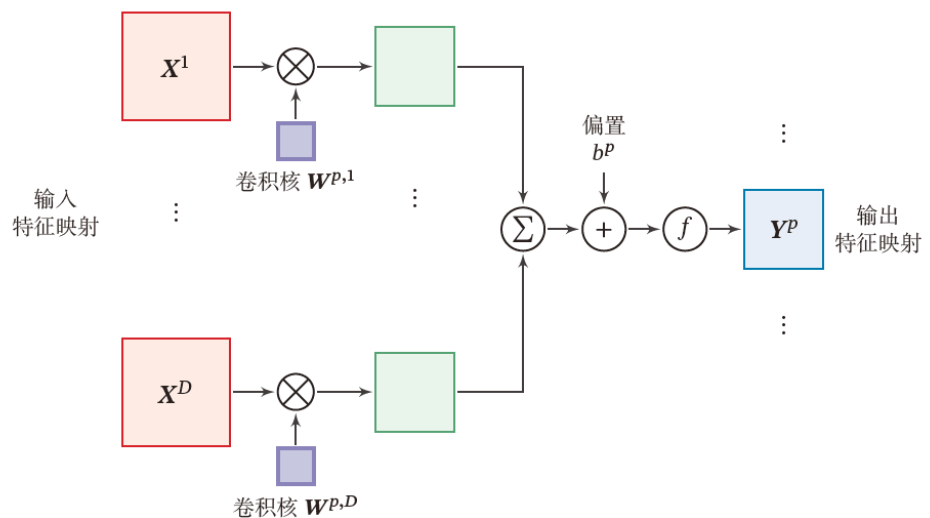


图 5.7 卷积层中从输入特征映射组 $\mathbf{X}$ 到输出特征映射 $\mathbf{Y}^p$ 的计算示例

○ 卷积核尺寸及复杂度

■ 卷积层参数

D通道卷积和一个 b ，输出P个特征

$$P \times D \times (U \times V) + P$$

- 卷积时间复杂度

若输入为 $W_i \times H_i \times M$

输出为 $W_o \times H_o \times N$

卷积核大小为 D_k

有计算量 $D_k^2 \times M \times W_o \times H_o \times N$

- 卷积变种

在卷积的标准定义基础上，还可以引入卷积核的滑动步长和零填充来增加卷积的多样性，可以更灵活地进行特征抽取。

步长 (Stride) 是指卷积核在滑动时的时间间隔。

零填充 (Zero Padding) 是在输入向量两端进行补零。

假设卷积层的输入神经元个数为 M ，卷积大小为 K ，步长为 S ，在输入两端各填补 P 个 0，那么该卷积层的神经元数量为 $(M - K + 2P)/S + 1$ 。

一般常用的卷积有以下三类：

- 窄卷积

步长 $S=1$ ，两端不补零 $P=0$ ，卷积后输出长度为 $M-K+1$

- 宽卷积

步长 $S=1$ ，两端不补零 $P=0$ ，卷积后输出长度为 $M-K+1$

- 等宽卷积

步长 $S=1$ ，两端补零 $P=\frac{K-1}{2}$ ，卷积后输出长度为 M

- 卷积仿射变换形式

$$C = ZX = \begin{bmatrix} w_{11} & w_{12} & 0 & w_{21} & w_{22} & 0 & \cdots & 0 \\ 0 & w_{11} & w_{12} & 0 & w_{21} & w_{22} & \cdots & 0 \\ \vdots & & & & & & & \\ 0 & 0 & 0 & 0 & w_{11} & w_{12} & \cdots & w_{22} \end{bmatrix}_{4 \times 9} X_{9 \times 1}$$

$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} \quad X = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}$$

- 1×1 卷积核作用

1×1 卷积为全连接层的另一表示形式，用来减少或增加特征映射的深度。如果输入特征映射之间存在冗余信息， 1×1 的卷积相当于先进行一次特征抽取。

- 循环神经网络

- 长短期记忆网络 LSTM

LSTM 为引入了门控机制控制信息传递的循环神经网络模式，同时强调了内部状态和外部状态差别，用于计算信息流。有内外部状态基于以下两个机制计算：

$$\begin{aligned} c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\ h_t &= o_t \odot \tanh(c_t) \end{aligned}$$

其中通过几个门控模块控制状态的转移：

- 输入门 i_t ：控制当前候选状态 $\tilde{c}^t$ 有多少保存

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

- 输出门 o_t ：控制当前内部状态 c_t 到外部状态 h_t 的输出

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

-
- 遗忘门 f_t ：控制上一时刻内部状态 c_{t-1} 的遗忘

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

总流程：

1. 利用上一时刻外部状态 h_{t-1} 和当前时刻输入 x_t 计算三个门及候选状态 $\tilde{c}^t$
2. 结合遗忘门 f_t 和输入门 i_t 更新记忆单元 c_t
3. 结合输出门 o_t 将内部状态信息输出到外部状态 h_t

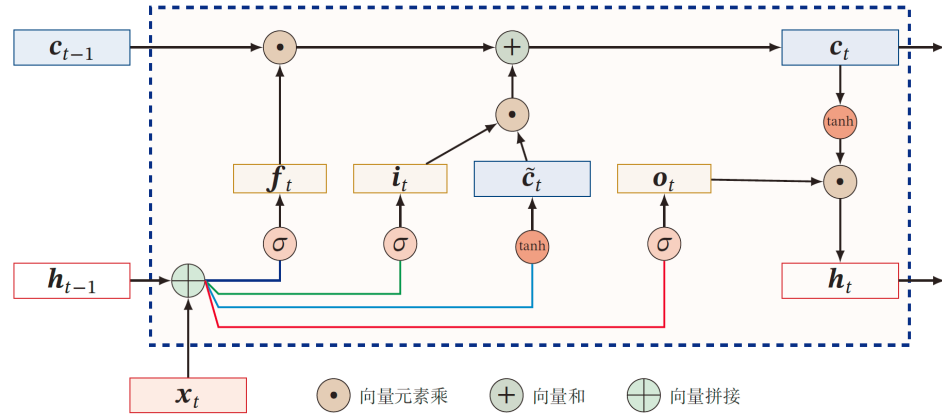


图 6.7 LSTM 网络的循环单元结构

○ 长程依赖问题

循环神经网络在学习过程中的主要问题是由于梯度爆炸或消失问题造成的长时间间隔状态难以建模。因此我们需要改进网络性质来解决长程依赖问题。

■ 梯度爆炸

一般通过权重衰减或梯度截断来避免。权重衰减是通过给参数增加 ℓ_1 或 ℓ_2 范数的正则化项来限制参数的取值范围，从而使得 $\gamma \leq 1$ 。梯度截断是另一种有效的启发式方法，当梯度的模大于一定阈值时，就将它截断成为一个较小的数。

■ 梯度消失

使用一些优化技巧，或改变模型，加入门控。

可以使用类似残差连接的改进策略：

$$h_t = h_{t-1} + g(x_t, h_{t-1}; \theta)$$

但这样的改进容易引入梯度爆炸问题和记忆容量问题。

○ 参数学习

随时间反向传播算法BPTT

将循环神经网络看作一个展开的多层前馈网络，其中“每一层”对应循环网络中的“每个时刻”。前馈网络中所有层的参数共享，参数的真实梯度是所有“展开层”的参数梯度之和。

依据公式前向传播：

$$\mathbf{z}_k = \mathbf{U}\mathbf{h}_{k-1} + \mathbf{W}\mathbf{x}_k + \mathbf{b}$$

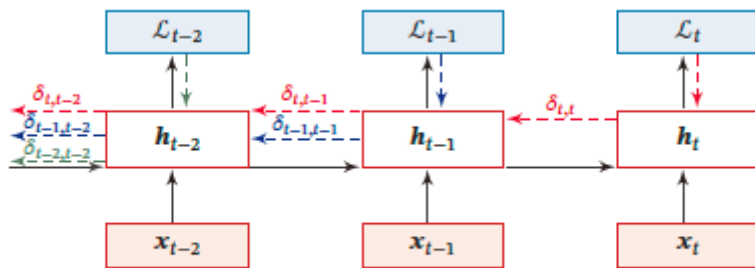


图 6.6 误差项随时间反向传播算法示例

$$\frac{\partial \Omega}{\partial \mathbf{U}} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^\top$$

$$\frac{\partial \Omega}{\partial \mathbf{W}} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{x}_k^\top$$

$$\frac{\partial \Omega}{\partial \mathbf{b}} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k}$$

- 网络优化与正则化
 - 优化算法

表 7.1 神经网络常用优化方法的汇总

类别		优化算法
学习率	固定衰减学习率	分段常数衰减、逆时衰减、(自然)指数衰减、余弦衰减
	周期性学习率	循环学习率、SGDR
	自适应学习率	AdaGrad、RMSprop、AdaDelta
梯度估计修正		动量法、Nesterov 加速梯度、梯度截断
综合方法		Adam ≈ 动量法 + RMSprop

- 参数初始化

参数初值的选取关系到了网络的优化效率与泛化能力

- 预训练初始化
- 随机初始化
- 固定值初始化

基于固定方差的参数初始化

从一个固定均值（通常为0）和方差 σ^2 的分布中采样来生成参数的初始值

- 高斯分布初始化：使用一个高斯分布 $\mathcal{N}(0, \sigma^2)$ 对每个参数进行随机初始化。
- 均匀分布初始化：在一个给定的区间 $[-r, r]$ 内采用均匀分布来初始化参数。假设随机变量 x 在区间 $[a, b]$ 内均匀分布，则其方差为 $D(x) = \frac{(b-a)^2}{12}$ 。有满足 $D(x) = \sigma^2$ 时 r 的取值为 $r = \sqrt{3\sigma^2}$

- 超参数优化

在神经网络中除可学习参数之外的参数。常见的超参数有以下三类：

1. 网络结构，包括神经元之间的连接关系、层数、每层的神经元数量、激活函数的类型等。
2. 优化参数，包括优化方法、学习率、小批量的样本数量等。

3. 正则化系数。

超参数优化是比较复杂的组合优化问题，同时优化的时间代价极高，因此多数优化方法在超参数优化上难以使用，比较简单的方法有网格搜索等。

网格搜索（Grid Search）是一种通过尝试所有超参数的组合来寻找合适一组超参数配置的方法。假设总共有 K 个超参数，第 k 个超参数的可以取 m_k 个值。那么总共的配置组合数量为 $m_1 \times m_2 \times \dots \times m_K$ 。如果超参数是连续的，可以将超参数离散化，选择几个“经验”值。比如学习率 α ，我们可以设置 $\alpha \in \{0.01, 0.1, 0.5, 1.0\}$ 。一般而言，对于连续的超参数，我们不能按等间隔的方式进行离散化，需要根据超参数自身的特点进行离散化。网格搜索根据这些超参数的不同组合分别训练一个模型，然后测试这些模型在开发集上的性能，选取一组性能最好的配置。

- 注意力机制与外部记忆

- 记忆增强网络

引入额外的外部记忆单元进行储存和读取

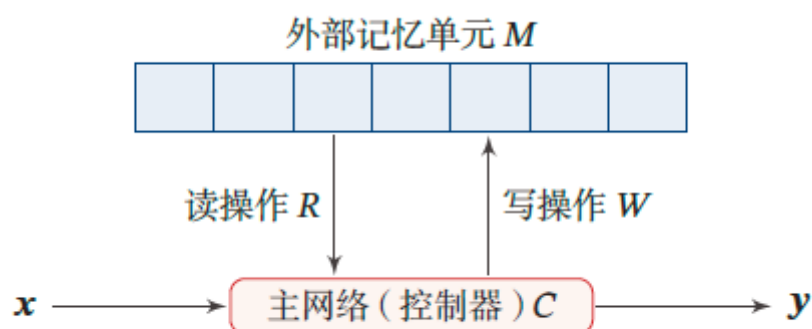


图 8.6 记忆网络的典型结构

1. 主网络 C ：也称为控制器（Controller），负责信息处理，以及与外界的交互（接受外界的输入信息并产生输出到外界）。主网络还同时通过读写模块和外部记忆进行交互。
2. 外部记忆单元 M ：外部记忆单元用来存储信息，一般可以分为很多记忆片段（Memory Segment），这些记忆片段按照一定的结构来进行组织。记忆片段一般用向量来表示，外部记忆单元可以用一组向量 $M = [m_1, \dots, m_N]$ 来表示。这些向量的组织方式可以是集合、树、栈或队列等。大部分信息存储于外部记忆中，不需要全时参与主网络的运算。
3. 读取模块 R ：根据生成查询向量 q_r ，从外部记忆单元中读取相应的信息 $r = R(M, q_r)$ 。
4. 写入模块 W ：根据生成查询向量 q_w 和要写入的信息 a 来更新外部记忆 $M = W(M, q_w, a)$ 。

这种结构化的外部记忆是带有地址的，由查询方法进行寻址

$$r = \sum_{n=1}^N \alpha_n m_n$$
$$\alpha_n = \text{softmax}(s(m_n, q_r))$$

其中 q_r 是主网络生成的查询向量， $s(\cdot, \cdot)$ 为打分函数。

通过引入外部记忆，可以在少量增加网络参数的条件下可以大幅增加网络容量。可以将注意力机制看作一个接口，将信息的存储与计算分离。

- 端到端记忆网络&神经图灵机

异同（考点）：

- 不同：

端到端的外部记忆单元只可读，而神经图灵机的外部记忆单元可以读写

端到端使用多跳方式进行多轮交互

神经图灵机具有增加和删除功能

- 相同：

均用于保存旧的外部记忆

- 无监督学习

- 主成分分析PCA

主成分分析是一种最常用的数据降维方法，使得在转换后的空间中数据的方差最大。从而最大化数据的差异性保留原始信息。

PCA方法希望将一组D维样本投影到一维空间，取投影向量 w 投影表示，有投影样本方差为

$$\begin{aligned}\sigma(X; w) &= \frac{1}{N} \sum_{n=1}^N (w^\top x^{(n)} - w^\top \bar{x})^2 \\ &= \frac{1}{N} \sum_{n=1}^N (w^\top X - w^\top \bar{X})(w^\top X - w^\top \bar{X})^\top \\ &= w^\top \Sigma w\end{aligned}$$

其中 $\Sigma = \frac{1}{N}(X - \bar{X})(X - \bar{X})^\top$ 为原始样本协方差矩阵，通过拉格朗日法转化无约束优化问题，可以发现取极值时，投影向量 w 为协方差矩阵的特征值，同时特征向量为投影后样本方差。因此可以通过矩阵特征值分解来求解主成分分析。对应投影至D'维空间，则依次保留较大特征值并选取对应投影矩阵。

主成分分析是一种无监督学习方法，可以作为监督学习的数据预处理方法，用来去除噪声并减少特征之间的相关性，但是它并不能保证投影后数据的类别可分性更好。

- 模型独立学习

- 集成学习

bagging：通过随机构造训练样本、随机选择特征等方法来提高每个基模型的独立性，代表性方法有Bagging 和随机森林等。

boosting：按照一定的顺序来先后训练不同的基模型，每个模型都针对前序模型的错误进行专门训练。根据前序模型的结果，来调整训练样本的权重，从而增加不同基模型之间的差异性。